WEEK 1 PROGRESS REPORT

SHARED L2 CACHE DESIGN SPACE EXPLORATION USING GEM5

Individual Role: System Verification and Performance Analysis

Reporting Period: Week 1

WEEKLY HIGHLIGHT

Main Achievement

Established the initial 4-core RISC-V shared-L2 cache baseline configuration in gem5 using the `RiscvMinorCPU` model. The configuration defines private L1 instruction and data caches for each core, a shared L2 cache, an `L2XBar` interconnect, and a DDR4 memory subsystem.

Completed Tasks

- Constructed the initial multicore cache hierarchy with four `RiscvMinorCPU` instances.
- Configured private L1I/L1D caches and a shared L2 cache using the baseline parameters.
- Defined the interconnect and memory subsystem connecting the shared L2 cache to DDR4 memory.
- Configured the baseline MSHR resources with 8 entries per private L1 cache and 32 entries in the shared L2 cache for subsequent performance verification.

Current Status

The baseline configuration has been constructed, but workload execution and simulation-based verification have not yet been completed. No performance measurements are reported at this stage.

Next Steps

- Integrate a RISC-V executable workload into the baseline configuration.
- Run the simulation and verify the normal termination of gem5.
- Inspect `stats.txt` to confirm cache and memory activity and establish the initial performance baseline.

1. Objective

The objective of the first stage is to establish a baseline gem5 system configuration for investigating the performance of a shared L2 cache in a multicore RISC-V system.

The baseline provides a common architectural configuration for subsequent experiments on cache parameters and non-blocking cache behavior. Before conducting design-space exploration, the configuration must first be integrated with a workload and verified through actual simulation.

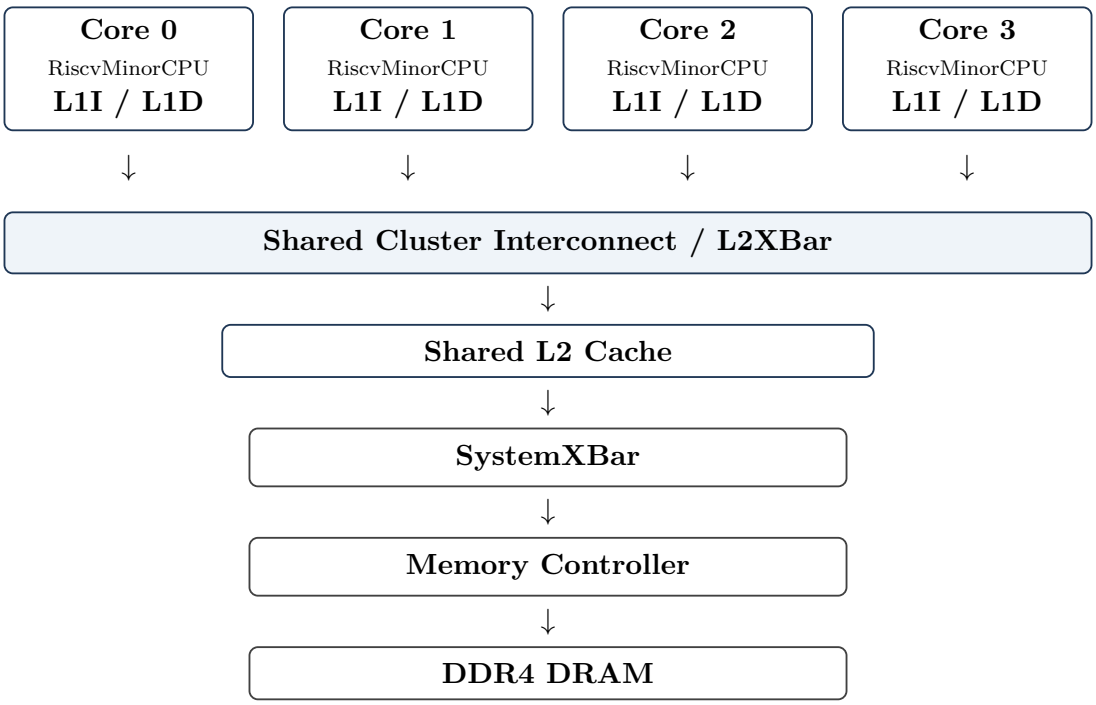
The current stage focuses on constructing the system hierarchy and specifying its principal parameters, rather than measuring performance or drawing conclusions about cache behavior.

2. Baseline Configuration

2.1 System Architecture

The initial system consists of four RISC-V cores modeled using `RiscvMinorCPU`. Each core has a private instruction cache (L1I) and data cache (L1D). The private L1 caches connect to a shared L2 cache through an `L2XBar` interconnect. The shared L2 cache connects to the memory subsystem through the system bus and a memory controller configured for DDR4 memory.

The resulting architectural hierarchy is:



Hình 1: Logical architectural flow of the baseline four-core shared-L2 system.

This configuration establishes the intended cache hierarchy for the shared-L2 investigation. The `L2XBar` is used as the modeled cluster interconnect and should not be interpreted as a cycle-accurate implementation of a physical network-on-chip.

2.2 Baseline Parameters

Table 1 summarizes the principal parameters defined in the initial configuration.

Bång 1: Initial gem5 baseline parameters

Component	Parameter	Configured Value
CPU	CPU model	4 × `RiscvMinorCPU`
L1 cache	Organization	Private L1I and L1D per core
L1 cache	Capacity	32 KiB per L1 cache
L1 cache	Associativity	4-way
L1 cache	Tag/data/response latency	1 / 1 / 1 cycles, as configured
L1 cache	MSHR entries	8 per L1 cache
L2 cache	Organization	One shared L2 cache
L2 cache	Capacity	1 MiB
L2 cache	Associativity	16-way
L2 cache	Tag/data/response latency	14 / 14 / 1 cycles, as configured
L2 cache	MSHR entries	32
Interconnect	Model	`L2XBar`
Interconnect	Width	16 bytes (128 bits)
Interconnect	Frontend / forward / response latency	2 / 2 / 2 cycles, respectively
Memory	Memory model	DDR4-2400, 8×8 configuration
System	Address range	512 MiB
Clock	Configured frequency	1 GHz
Cache line	Block size	64 bytes

The parameters above describe the configured model. They do not, by themselves, establish measured access latencies or simulation performance.

2.3 MSHR Configuration

The baseline specifies non-blocking cache resources for both the private L1 caches and the shared L2 cache.

Each private L1 cache is configured with eight MSHR entries, while the shared L2 cache is configured with 32 MSHR entries.

An MSHR (Miss Status Holding Register) tracks an outstanding cache miss while the requested cache line is being serviced by a lower level of the memory hierarchy. Multiple MSHR entries allow the cache to maintain several outstanding misses concurrently and therefore support memory-level parallelism.

The 32-entry MSHR configuration belongs specifically to the shared L2 cache; it is not a system-wide MSHR pool. This value is retained as the baseline setting so that future cache-configuration experiments can isolate the effect of L2 organization without simultaneously

changing the number of available L2 MSHR entries.

At the current stage, MSHR occupancy, saturation, stall behavior, and multi-core contention have not yet been measured. These behaviors require workload-driven simulation and analysis of the generated gem5 statistics.

3. Individual Work Completed

3.1 Construction of the Baseline Hierarchy

The initial Python configuration was organized to define the four-core RISC-V system, private L1 caches, shared L2 cache, interconnect, and memory subsystem.

The main implementation objective was to establish the connections between the modeled components and specify their baseline parameters. This creates the initial configuration required for subsequent workload integration and verification.

3.2 Identification of Verification Requirements

The baseline must be verified through simulation rather than configuration inspection alone. The planned verification procedure includes:

1. Confirm that gem5 accepts and initializes the configuration.
2. Integrate a compatible RISC-V executable workload.
3. Confirm that the simulation starts and executes the workload.
4. Verify that the simulation terminates normally.
5. Inspect `stats.txt` for evidence of instruction execution, cache activity, and memory transactions.
6. Check the consistency of the reported statistics before using them in performance analysis.

At the current stage, this procedure has been defined as the next verification task. The steps requiring an actual workload and simulation output remain pending.

4. Current Limitations and Pending Work

The current implementation represents the initial configuration stage rather than a completed experimental baseline. The following items remain outstanding:

- **Workload integration:**

  A RISC-V executable has not yet been integrated into the current baseline run.

- **Simulation verification:**

  Normal workload execution and simulation termination have not yet been demonstrated.

- **Statistics collection:**

  Cache miss counts, miss rates, memory traffic, execution cycles, and other performance metrics have not yet been collected from a workload run.

- **MSHR analysis:**

  MSHR occupancy, saturation, stall behavior, and possible multi-core contention have not yet been evaluated.

- **AMAT evaluation:**

  No workload-based AMAT result is reported because the required measurements are not yet available.

In addition, `RiscvMinorCPU` is a gem5 CPU model and should not be treated as a cycle-accurate model of the actual CVA6 RTL implementation. Any correspondence between the modeled CPU and CVA6 must be stated as a modeling approximation.

The cache and interconnect latency values reported in this document are configured simulator parameters. They should not be interpreted as measured end-to-end access latency until workload-driven timing behavior has been verified.

5. Conclusion

The first stage established the initial architecture and principal parameters of a four-core RISC-V shared-L2 cache system in gem5. The configuration includes private L1 instruction and data caches, a shared 1 MiB L2 cache, an `L2XBar` interconnect, a system memory bus, a memory controller, and a DDR4 memory subsystem.

The private L1 caches are configured with eight MSHR entries each, while the shared L2 cache uses a fixed 32-entry MSHR baseline. The L2 MSHR configuration will later be analyzed under multicore workloads to identify possible saturation and contention effects.

This work provides the structural foundation for the planned performance study. However, the baseline has not yet been verified through workload execution. The next milestone is to run a compatible workload, inspect the simulation output, and establish a measured reference point before proceeding to cache design-space exploration.